

Module 3 - Database Integration - PostgreSQL, MySQL & MongoDB

Given data - Common Scenario: Student Course Registration System

HANDS-ON 4 - Query Optimisation — Indexes, EXPLAIN & the N+1 Problem

Step 48

SQL

EXPLAIN QUERY PLAN

```
SELECT s.first_name,  
       s.last_name,  
       c.course_name  
FROM enrollments e  
JOIN students s  
ON s.student_id = e.student_id  
JOIN courses c  
ON c.course_id = e.course_id  
WHERE s.enrollment_year = 2022;
```

MY EXAMPLE TABLE

Input

Run SQL

```
EXPLAIN QUERY PLAN
SELECT s.first_name,
       s.last_name,
       c.course_name
FROM enrollments e
JOIN students s
ON s.student_id = e.student_id
JOIN courses c
ON c.course_id = e.course_id
WHERE s.enrollment_year = 2022;
```

Output

id	parent	notused	detail
6	0	0	SCAN e
8	0	0	SEARCH s USING INDEX sqlite_autoindex_students_1 (student_id=?)
17	0	0	SEARCH c USING INDEX sqlite_autoindex_courses_1 (course_id=?)

49

EXAMPLE TABLE

```
-- Query Plan Analysis
-- The query plan shows a table scan (SCAN) on one or more tables.
-- Since no indexes were added yet, SQLite scans the table to locate matching rows.
```

Output

SQL query successfully executed. However, the result set is empty.

Step 50

Add a comment like:

```
-- Estimated rows are determined by
SQLite's query planner.
-- The query currently performs a full scan
before optimisation.
-- This serves as the baseline performance
measurement.
```

EXPECTED OUTPUT

Input

SQL

🌙

☰

Run SQL

```

EXPLAIN QUERY PLAN
SELECT s.first_name,
       s.last_name,
       c.course_name
FROM enrollments e
JOIN students s
ON s.student_id = e.student_id
JOIN courses c
ON c.course_id = e.course_id
WHERE s.enrollment_year = 2022;

```

Output

id	parent	notused	detail
6	0	0	SCAN e
8	0	0	SEARCH s USING INDEX sqlite_autoindex_students_1 (student_id=?)
17	0	0	SEARCH c USING INDEX sqlite_autoindex_courses_1 (course_id=?)

Task 2: Add Indexes and Compare Plans

Goal: Create indexes and observe their impact on query plans

Step 51: Create index on enrollment_year

```
CREATE INDEX idx_students_enrollment_year
```

ON students(enrollment_year);

MY EXAMPLE TABLE

Input

```
CREATE INDEX idx_students_enrollment_year
ON students(enrollment_year);
```

Run SQL

Output

SQL query successfully executed. However, the result set is empty.

Step 52: Create composite UNIQUE index

```
CREATE UNIQUE INDEX
idx_enrollments_student_course
ON enrollments(student_id, course_id);
```

Input

```
CREATE UNIQUE INDEX idx_enrollments_student_course
ON enrollments(student_id, course_id);
```

Run SQL

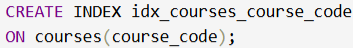
Output

SQL query successfully executed. However, the result set is empty.

Step 53: Create index on course_code

```
CREATE INDEX idx_courses_course_code  
ON courses(course_code);
```

Input



```
CREATE INDEX idx_courses_course_code  
ON courses(course_code);
```

Output

SQL query successfully executed. However, the result set is empty.

Step 54: Re-run EXPLAIN QUERY PLAN

```
EXPLAIN QUERY PLAN  
SELECT s.first_name,  
       s.last_name,  
       c.course_name  
FROM enrollments e  
JOIN students s  
ON s.student_id = e.student_id  
JOIN courses c  
ON c.course_id = e.course_id  
WHERE s.enrollment_year = 2022;
```

Input

EXPLAIN QUERY PLAN
SELECT s.first_name,
s.last_name,
c.course_name
FROM enrollments e
JOIN students s
ON s.student_id = e.student_id
JOIN courses c
ON c.course_id = e.course_id
WHERE s.enrollment_year = 2022;

Run SQL

Output

id	parent	notused	detail
6	0	0	SEARCH s USING INDEX idx_students_enrollment_year (enrollment_year=?)
11	0	0	SEARCH e USING COVERING INDEX idx_enrollments_student_course (student_id=?)
17	0	0	SEARCH c USING INDEX sqlite_autoindex_courses_1 (course_id=?)

Step 55: Partial Index

CREATE INDEX idx_enrollments_null_grade

ON enrollments(student_id)

WHERE grade IS NULL;

Input

CREATE INDEX idx_enrollments_null_grade
ON enrollments(student_id)
WHERE grade IS NULL;

Run SQL

Output

SQL query successfully executed. However, the result set is empty.

Expected Outcome: EXPLAIN output shows an Index Scan on at least one table. The composite UNIQUE index prevents duplicate enrollment inserts

Task 3: Identify and Fix the N+1 Problem Goal: Recognise and resolve the most common ORM performance anti-pattern

Step 56: N+1 Problem Example

PYTHON CODE

```
import sqlite3
```

```
conn = sqlite3.connect("college_db.db")
```

```
cursor = conn.cursor()
```

```
query_count = 0
```

```
cursor.execute("SELECT * FROM enrollments")
```

```
enrollments = cursor.fetchall()
```

```
query_count += 1
```

```
for enrollment in enrollments:
```

```
    student_id = enrollment[1]
```



```
    cursor.execute(
        "SELECT first_name, last_name FROM students
WHERE student_id=?",
        (student_id,)
    )
    cursor.fetchone()
    query_count += 1
```

```
print(f"{query_count} queries executed")
```

Step 57: Fix using JOIN

```
import sqlite3
```

```
conn = sqlite3.connect("college_db.db")
```

```
cursor = conn.cursor()
```

```
query_count = 0
```

```
cursor.execute("""
SELECT e.enrollment_id,
       s.first_name,
```

```
s.last_name,  
e.course_id,  
e.grade  
FROM enrollments e  
JOIN students s  
ON e.student_id = s.student_id  
""")
```

```
rows = cursor.fetchall()  
query_count += 1
```

```
print(f"{query_count} query executed")
```

Step 58: Compare execution time

```
import sqlite3  
import time
```

```
conn = sqlite3.connect("college_db.db")  
cursor = conn.cursor()
```

```
start = time.time()
```

```
cursor.execute("""  
SELECT e.enrollment_id,  
       s.first_name,  
       s.last_name,  
       e.course_id  
FROM enrollments e  
JOIN students s  
ON e.student_id = s.student_id  
""")
```

```
cursor.fetchall()
```

```
end = time.time()
```

```
print("Execution Time:", end - start)
```

Step 59: Documentation Comment

N+1 Problem Analysis

For N enrollments:

1 query retrieves all enrollments.

N additional queries retrieve related student data.

Total queries = N + 1.

Example:

10,000 enrollments

= 1 + 10,000

= 10,001 database queries.

JOIN-based approach:

Only 1 query is executed regardless of N.

This significantly improves performance.

